

Unit Testing in C++

Writing a Test Driver That Is Actually Useful

COEN 244 – Programming Methodology II

Fall 2026

What is a unit test?

Definition

A **unit test** is a small piece of code that calls one function with known inputs and checks that the result is what you expected.

Every unit test has the same three parts:

```
// 1. Arrange -- set up the input  
std::string plaintext = "coen244";  
// 2. Act -- call the function once  
std::string actual = encrypt(plaintext, 13);  
// 3. Assert -- compare against what you expected  
check("encrypt basic", actual, "pbra244");
```

Where does the expected answer come from?

You work it out yourself

By hand, from the specification. Not by running the function.

The rule

If you get the expected value by calling the function you are testing, the test always passes and proves nothing.

This is why writing tests is real work. You have to know what the answer *should* be.

Our example: a Caesar cipher

Each letter shifts forward by a fixed amount and wraps at z. Digits and punctuation are unchanged.

```
std::string encrypt(const std::string& plaintext, int shift);  
std::string decrypt(const std::string& ciphertext, int shift);
```

Working out encrypt("coen244", 13) by hand:

Letter	Position	+13 mod 26	Result
c	2	15	p
o	14	1	b
e	4	17	r
n	13	0	a

So the answer is "pbra244". That is our expected value.

The test driver we have now

```
int main() {  
    std::string testResult =  
        (testEncrypt() == "pbra244") ? "pass" : "fail";  
    std::cout << testResult << std::endl;  
    testResult =  
        (testDecrypt() == "coen244") ? "pass" : "fail";  
    std::cout << testResult << std::endl;  
    testMultipleCases();  
    return 0;  
}
```

It runs. It prints something.
Let us look at what it prints.

Three problems

The output is:

```
pass  
fail
```

- ❶ Which test failed? You have to count lines and look at the source. Fine with two tests. Useless with twenty.
- ❷ What did it actually return? You know it was wrong. You do not know what it was. That is the one thing you needed.
- ❸ main always returns 0. Even when tests fail, the program reports success.

Key point: problem 3 is the serious one

A test suite that cannot fail is decoration

Printing `fail` and then returning 0 gives you a program that looks careful and reports success while the code is broken.

The **exit code** is how other programs find out whether your tests passed:

- build scripts
- automated marking

Nobody reads your `cout` output but you.

Fix all three with one helper function

```
int testsRun = 0;
int testsFailed = 0;
void check(const std::string& testName,
           const std::string& actual,
           const std::string& expected) {
    ++testsRun;
    if (actual == expected) {
        std::cout << "[ PASS ] " << testName << "\n";
    } else {
        ++testsFailed;
        std::cout << "[ FAIL ] " << testName << "\n"
                  << "          expected: " << expected << "\n"
                  << "          actual:    " << actual << "\n";
    }
}
```

Fifteen lines. No library, no framework.

The complete test driver

```
int main() {
    check("encrypt basic",    encrypt("coen244", 13), "pbra244");
    check("decrypt basic",    decrypt("pbra244", 13), "coen244");
    check("digits unchanged", encrypt("244", 13),    "244");
    check("empty string",     encrypt("", 13),        "");
    std::cout << "\n" << (testsRun - testsFailed)
              << " / " << testsRun << " tests passed\n";
    return (testsFailed == 0) ? 0 : 1;
}
```

Each test is one line: a name, a call, an expected value.
Adding a test means adding a line.

What the output looks like now

```
[ PASS ] encrypt basic
[ FAIL ] decrypt basic
        expected: "coen244"
        actual:   "cnem244"
[ PASS ] digits unchanged
[ PASS ] empty string

3 / 4 tests passed
```

Two letters are wrong: o became n, and n became m. Both are exactly the letters that had to **wrap around** the start of the alphabet — so the bug is in the wrap, and you found it **without opening the debugger**.

Compiling and running it

No build system needed. One command:

```
g++ -std=c++17 -Wall testDriver.cpp CaesarCipherEncryption.cpp -o testDriver
./testDriver
```

Check that the exit code is right:

```
./testDriver
echo $?           # Linux / macOS

testDriver.exe
echo %ERRORLEVEL% # Windows
```

You want 0 when all tests pass and 1 when any fail.

Choosing what to test

The mechanics are easy. Picking the inputs is the real skill.

Kind of input	Example	Why
Ordinary case	("coen244", 13)	the normal path
Empty input	("", 13)	a common crash site
No shift	("abc", 0)	catches off-by-one
Wrap-around	("z", 1)	the tricky boundary
Full cycle	("abc", 26)	should be unchanged
Not a letter	("2+4", 13)	the pass-through rule

Rule of thumb

Bugs live at the **boundaries**. If ("m", 13) works, ("n", 13) almost certainly works too — so test ("z", 1) instead.

Key point: tests do not prove correctness

Passing tests mean one thing only

Your code works *on the inputs you thought to try*.

Two consequences worth remembering:

- A green test run is a **lower bound** on quality, never a proof.
- Shift 13 is a bad choice for testing this cipher: encrypting twice gives back the original, so swapping encrypt and decrypt by mistake would go **undetected**. Add a test with a different shift.

What a real framework adds

Our check function works, but it only compares strings, and one crash stops the whole run. Frameworks like **GoogleTest** handle those cases.

```
#include <gtest/gtest.h>
TEST(CaesarCipher, EncryptBasic) {
    EXPECT_EQ(encrypt("coen244", 13), "pbra244");
}
```

Compare with what you just wrote:

Yours	GoogleTest
check(...)	EXPECT_EQ(...)
your counters and summary	the run summary
return testsFailed ? 1 : 0	RUN_ALL_TESTS()

Same ideas. You have already built the important part.

Summary

- A unit test is **Arrange, Act, Assert** — with the expected value worked out by you, not by the code.
- Printing `pass` or `fail` is not enough. Name the test, show expected against actual, and **return a non-zero exit code** when anything fails.
- Fifteen lines of `check()` gives you a real test suite. No framework required.
- Choosing good inputs is harder than writing the test code, and **boundaries are where the bugs are**.
- Passing tests do not prove your code is correct.